Universidad del Valle de Guatemala

Facultad de Ingeniería

Computación Paralela y Distribuida - Semestre 2, 2026



Proyecto 02 - Entrega del inciso 1

Cifrado y descifrado de archivos de texto

AES-128 con OpenSSL

Integrantes:

Daniel Estrada - 20853

Andre Pivaral - 23574

Sebastian Estrada - 21405

Guatemala, 14 de octubre de 2026

Índice

Introducción, fundamentos y diseño 3
Evidencia de compilación 4
Evidencia de cifrado 5
Evidencia de descifrado 6
Verificación y pruebas 7
Anexo 1: catálogo de funciones y bibliotecas 8
Discusión, conclusiones, recomendaciones y referencias 9

Introducción, fundamentos y diseño

Se adaptó el código del Laboratorio 03 para leer un archivo de texto, cifrar su contenido con una clave numérica recibida como parámetro y recuperar los mismos bytes usando esa clave. El objetivo de este inciso es demostrar lectura, cifrado, descifrado y exactitud del resultado.

Fundamentos. AES-128 utiliza claves de 128 bits (16 bytes) [1]. Se conserva el modo GCM del laboratorio, que combina cifrado y autenticación [2], mediante la interfaz EVP de OpenSSL [3]. El programa genera un nonce aleatorio de 12 bytes y conserva una etiqueta de 16 bytes. El texto descifrado solo se muestra y guarda después de verificar la autenticación.

Representación de la clave. `make_key` pone a cero los 16 bytes y coloca el entero en los últimos ocho, en orden big-endian. Para 123456, la clave es `0000000000000000000000000001e240`. Se preserva así la convención experimental del laboratorio; no se recorre todo el espacio de claves AES.

Archivo AESLAB1 (texto ASCII)	Contenido
Líneas 1 y 2	Identificador AESLAB1 y longitud original en bytes.
Líneas 3, 4 y 5	Nonce, etiqueta y contenido cifrado en hexadecimal; no incluye la clave ni el texto original.

Flujo de ejecución

Cifrar: validar argumentos → leer bytes del .txt → convertir clave → generar nonce → cifrar y obtener etiqueta → crear archivo AESLAB1 → mostrar original y cifrado.
Descifrar: validar argumentos → cargar AESLAB1 → convertir clave → descifrar y autenticar → si autentica, guardar y mostrar el texto; si falla, salir con error sin guardar texto.

Alcance: esta entrega desarrolla el inciso 1. La búsqueda por fuerza bruta, las estrategias MPI y las mediciones de speedup corresponden a los siguientes incisos; aquí no se presentan resultados de rendimiento.

Compilación y entorno

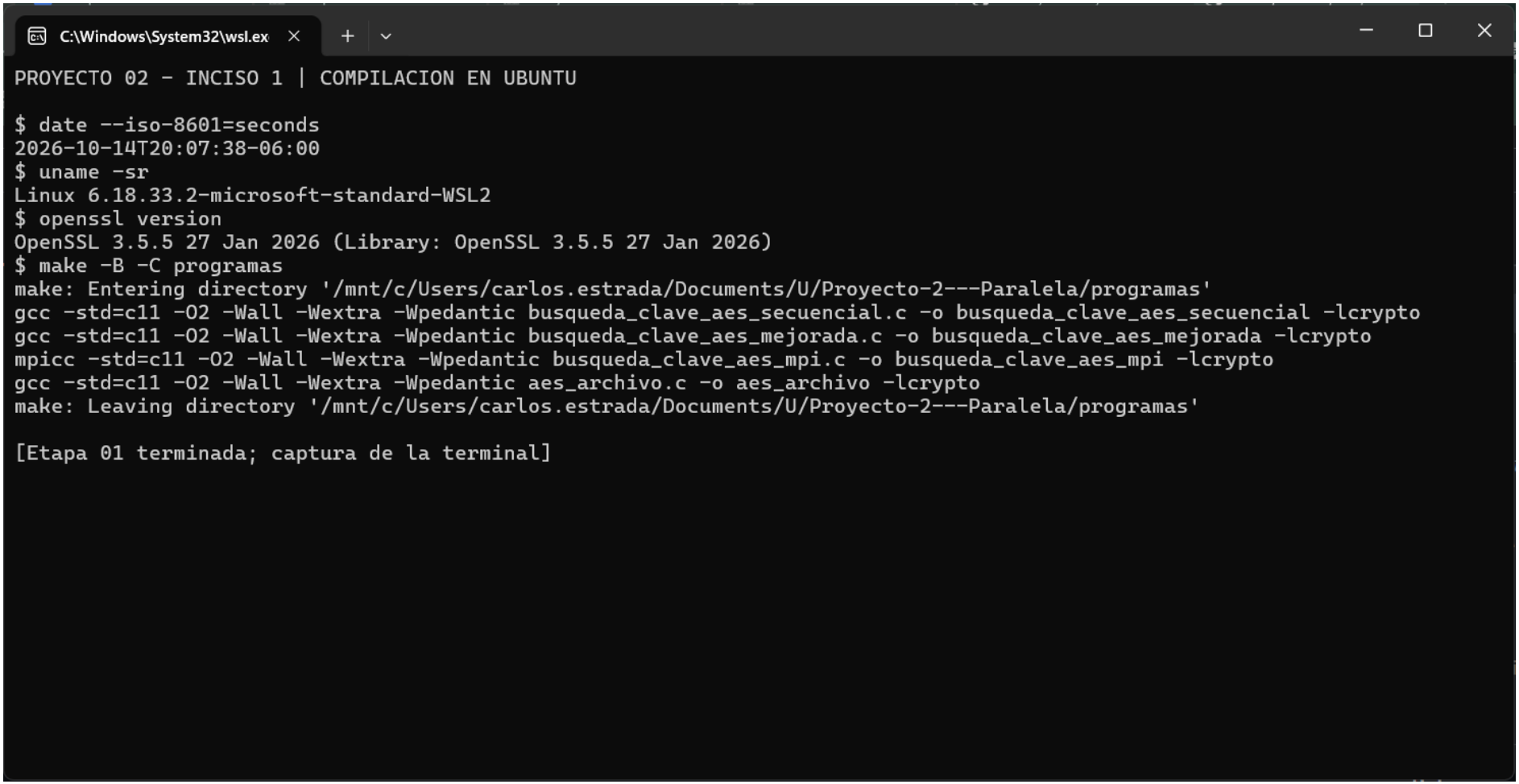


Figura 1. Captura directa de Windows Terminal ejecutando Ubuntu mediante WSL. Se muestran la fecha, el núcleo Linux, OpenSSL y los comandos de GCC y mpicc generados por Make.

Resultado: compilación de los cuatro ejecutables sin diagnósticos de error ni advertencias en esta ejecución. Las capturas preceden a la reorganización del repositorio. El ejecutable del inciso 1 ahora está en `proyecto02/build/aes_archivo`.

Lectura del .txt y cifrado

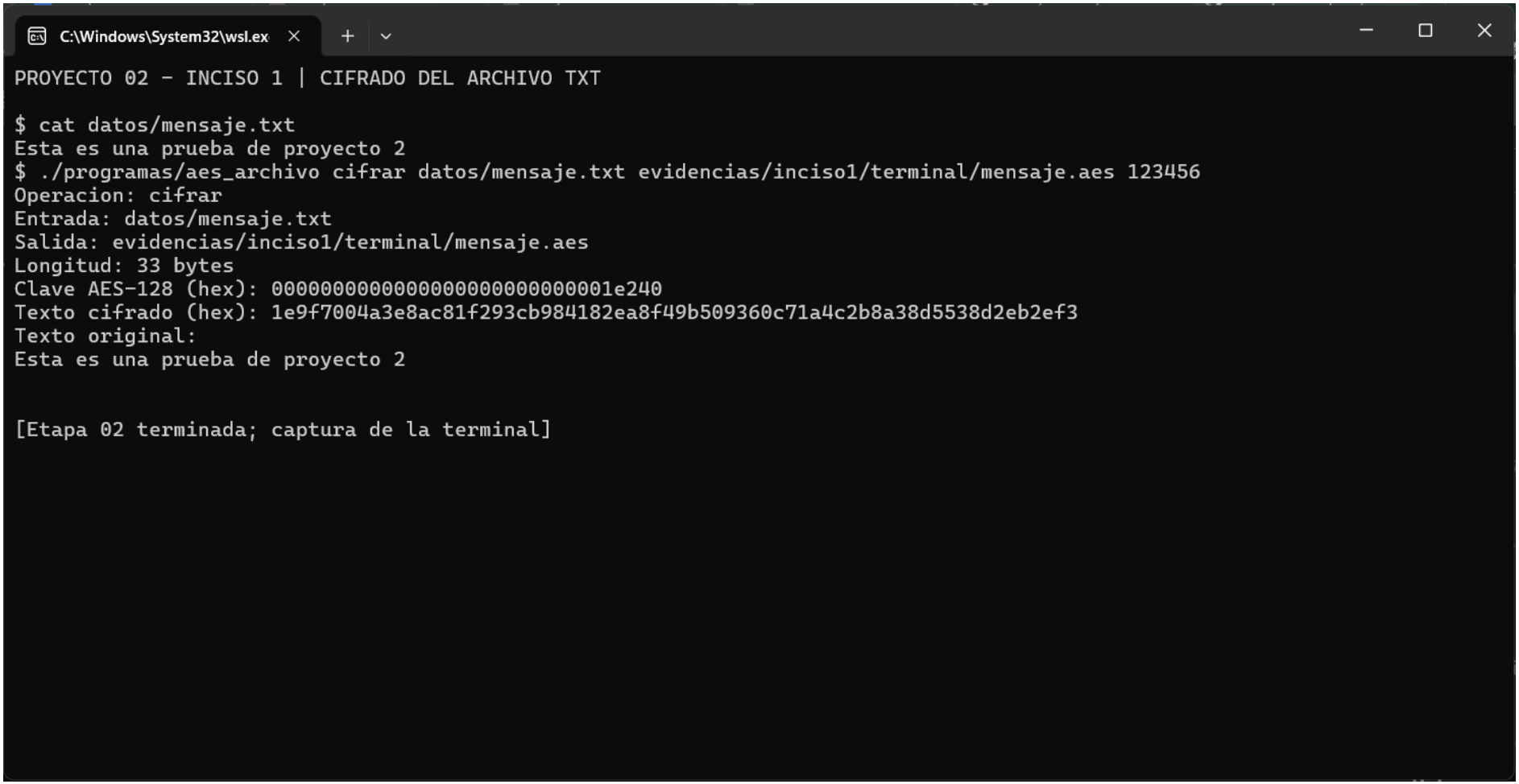


Figura 2. Lectura de datos/mensaje.txt y cifrado AES-128-GCM con el parámetro 123456. La salida muestra el texto original, la clave convertida a 16 bytes y el contenido cifrado en hexadecimal.

Resultado: se guarda mensaje.aes con 33 bytes de contenido cifrado. La frase tiene 32 bytes ASCII y el archivo de prueba añade un salto de línea final. El nonce aleatorio hace que una nueva ejecución pueda producir otro cifrado.

Descifrado y recuperación del texto

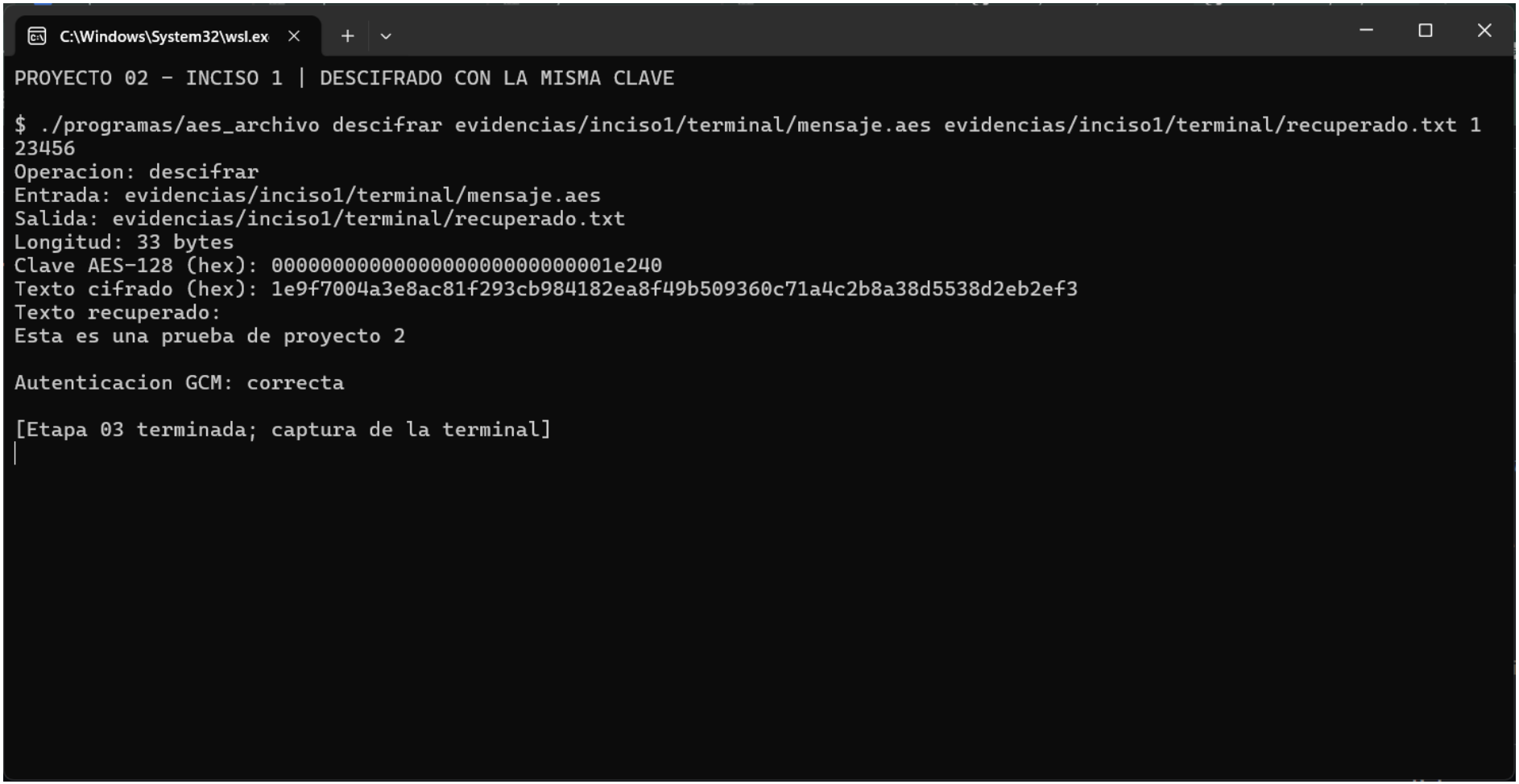


Figura 3. Descifrado del archivo generado con la misma clave numérica 123456. El comando completo es visible; la terminal ajusta la última parte de la clave a la línea siguiente.

Resultado: autenticación GCM correcta y escritura de recuperado.txt. El texto recuperado es la frase original y conserva su salto de línea final.

Igualdad de archivos y manejo de errores

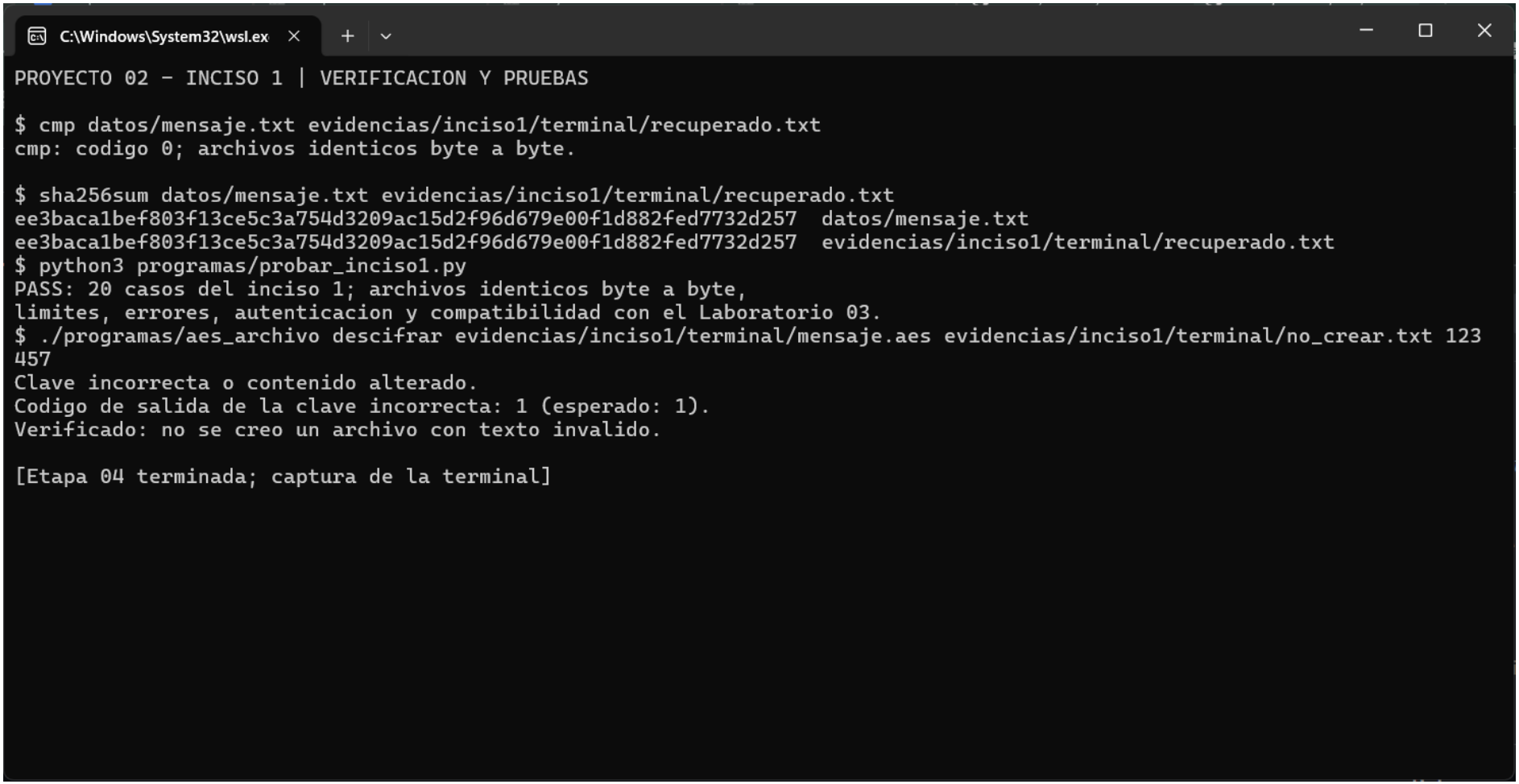


Figura 4. Comparación con cmp, hashes SHA-256, ejecución de los 20 casos de prueba y rechazo de la clave 123457.

Resultado: cmp retorna 0 y ambos hashes coinciden. Las 20 pruebas pasan. La clave incorrecta retorna 1 y no crea no_crear.txt. El hash del original y del recuperado es `ee3baca1bef803f13ce5c3a754d3209ac15d2f96d679e00f1d882fed7732d257`.

Anexo 1. Catálogo de funciones y bibliotecas

Función / archivo	Entradas	Salida y funcionamiento
main / aes_archivo.c	Operación, entrada, salida y clave.	0 si termina correctamente; 1 si falla. Coordina validación, AES, guardado y presentación.
read_text / aes_archivo.c	Ruta, buffer y puntero a longitud.	1 o 0; lee hasta 4096 bytes en binario, detecta exceso o error y conserva bytes exactos.
write_output / aes_archivo.c	Ruta, Dataset, texto y modo.	1 o 0; crea la salida de forma exclusiva, serializa AESLAB1 o guarda el texto autenticado.
parse_number / aes_comun.h	Texto decimal y puntero a uint64_t.	1 o 0; valida dígitos y rango; rechaza UINT64_MAX, reservado por la búsqueda heredada.
make_key / aes_comun.h	Entero de 64 bits y buffer de 16 bytes.	Completa el buffer con la clave AES usando ceros iniciales y orden big-endian.
encrypt_message / aes_comun.h	Contexto, clave, texto, longitud y Dataset con nonce.	1 o 0; cifra con EVP y almacena longitud, cifrado y etiqueta GCM.
try_key / aes_comun.h	Contexto, clave, Dataset y buffer de texto.	1 si autentica, 0 si no, -1 si falla la API. Borra el texto provisional ante autenticación fallida.
load_dataset / aes_comun.h	Ruta AESLAB1 y Dataset de destino.	1 o 0; valida identificador, longitud, campos hexadecimales y fin de archivo.
read_hex / aes_comun.h	Archivo, buffer y longitud.	1 o 0; convierte pares hexadecimales y comprueba el salto de línea.
print_hex / aes_comun.h	Flujo, bytes y longitud.	Escribe los bytes como pares hexadecimales y termina con un salto de línea.

Bibliotecas utilizadas: stdio.h para archivos y presentación; stdlib.h, errno.h y stdint.h para conversión y tipos; string.h para buffers; openssl/evp.h para AES, contextos y autenticación; openssl/rand.h para RAND_bytes. La compilación enlaza libcrypto. Python 3 ejecuta las pruebas. Open MPI pertenece a los programas heredados y no participa en el cifrado de este inciso.

Discusión, conclusiones y referencias

Discusión. Los retos principales fueron conservar el formato del laboratorio, mantener exactamente los bytes del .txt y evitar escribir texto provisional antes de autenticarlo. Se reutilizaron las rutinas AES y se añadieron lectura binaria, comandos de archivo y creación exclusiva de salidas. El cargador se adaptó para archivos vacíos.

Conclusiones. La demostración recuperó los 33 bytes originales y verificó su igualdad con cmp y SHA-256. Las pruebas cubrieron vacío, un byte, UTF-8, CRLF, 4096 bytes, claves grandes, claves inválidas, autenticación fallida, archivo alterado o truncado, lectura y escritura fallidas, protección de archivos y compatibilidad con el laboratorio. Se cumplieron los pasos funcionales de lectura, cifrado, descifrado y presentación requeridos para el inciso 1.

Limitaciones y recomendaciones. Se conserva el máximo de 4096 bytes por archivo y el dominio de claves numéricas de 0 a 18446744073709551614. Para archivos mayores se requerirá ampliar el manejo de datos. Para repetir las demostraciones se deben elegir archivos de salida nuevos. Los resultados de este inciso no permiten concluir nada sobre speedup ni sobre la eficiencia de la búsqueda MPI.

Referencias bibliográficas

[1] NIST. (2023). FIPS 197: Advanced Encryption Standard (AES). https://csrc.nist.gov/pubs/fips/197/final

[2] Dworkin, M. (2007). NIST SP 800-38D: Galois/Counter Mode (GCM) and GMAC. https://csrc.nist.gov/pubs/sp/800/38/d/final

[3] OpenSSL Project. (s. f.). EVP_CIPHER-AES. https://docs.openssl.org/3.0/man7/EVP_CIPHER-AES/

[4] Estrada, C. D. Laboratorio 03: código base. Commit e20c920fe0d105d2844889d7410563e12a3b3d1d. Repositorio del Laboratorio 03. Referencias consultadas el 14 de octubre de 2026.

Material suplementario y reproducción. El paquete de entrega incluye código fuente, Makefile, proyecto02/datos/mensaje.txt, pruebas, instrucciones y capturas PNG con su registro de sesión. Desde la raíz: `make` y `make test`. Los comandos de cifrado y descifrado se describen en README.md. Los archivos del ejemplo terminal y sus capturas se encuentran en evidencias/inciso1/.